

GuardSynth: Formal Specification and Verification of Behavioral Constraints for Enhanced CoC-Based VLM Learning

Author information pending confirmation

Affiliations and corresponding author to be confirmed

ABSTRACT Reasoning-based driving models such as Alpamayo connect scene understanding and action prediction through Chain of Causation (CoC). However, CoC explanations alone have limitations in learning correct action selection and constraint compliance. We present GuardSynth, a systematic method for developing behavioral constraints that augment CoC. The method identifies constraint candidates from CoC context, binds observations, supplied rules, and assumptions to specification fields, and represents applicability, prohibited actions, persistence, release, timing, and evidence using Evidence-Bound Lifecycle Contracts (EBLC). Restricted logical verification precedes controlled natural language (CNL) generation and insertion into the original CoC. We evaluate six synthetic action-selection tasks using Qwen3-VL-2B-Instruct with matched LoRA budgets within each task family and three seeds. Both our existing natural-language constraint path and the EBLC-derived CNL path achieve higher primary accuracy and lower violation rates than CoC alone. Temporal-task accuracy is 49.86%, 96.81%, and 98.06%, respectively, with violation rates of 19.17%, 1.39%, and 0.83%. Constraints are supplied during both training and evaluation. Separate specification-quality checks detect 144 of 168 injected errors, exposing shared evidence-specification errors as a remaining limitation. These results connect the behavioral benefits of explicit constraints with a traceable specification, verification, and language-generation process.

INDEX TERMS Behavioral constraints, Chain of Causation, controlled natural language, formal verification, vision-language models.

I. INTRODUCTION

Driving decisions require not only recognizing objects but also selecting appropriate actions from their relationships and changes in the scene. NVIDIA's Alpamayo-R1 addresses this problem with a vision-language-action (VLA) model that combines Chain of Causation (CoC) reasoning with trajectory prediction. It aligns causally connected explanations with driving behavior and uses supervised fine-tuning and reinforcement learning to improve reasoning quality and reasoning-action consistency. This approach illustrates learning from the reasoning that leads to an action as well as from the action itself. [1]

Related studies connect visual information and language-based reasoning in complementary ways. DriveLM organizes perception, prediction, and planning questions and answers into a graph for multistep driving reasoning. DriveVLM combines scene description, scene analysis, and hierarchical

planning to address complex driving situations. These studies provide a foundation for connecting scene interpretation to decisions and plans. Within this direction, we focus on the representation and verification of behavioral constraints supplied in learning data. [2], [3]

However, learning from CoC explanations alone has limitations in learning correct action selection and constraint compliance. Information explaining why an action is taken and information defining which actions are admissible play different roles. For example, "slow down to yield to the pedestrian" explains a reason, whereas "do not enter while the crossing zone is occupied" restricts the available actions. Different required waiting durations after clearance can change admissible entry times even for the same scene. We address the problem of augmenting explanation-based learning data with such explicit, situation-dependent constraints to improve action selection.

Our preceding manuscript, “Guard-Aware Trajectory Candidate Selection for Vision-Language Autonomous Driving: Safety-Constrained Chain-of-Causation,” proposes natural-language execution guards, their activation and release elements, and paired-contract evaluation over static, temporal, and maneuver tasks. The present paper builds on that constraint-enhancement approach rather than introducing it again. Its new focus is the development of those constraints: identifying their sources, binding them to typed fields and explicit semantics, checking a supported logical representation, and generating CNL for insertion into CoC. We reuse the task families and external candidate-scoring principle but report newly executed P0/P1/P2 experiments; earlier scores are not substituted for current results. [4]

This requires answering three questions. What scope and elements should behavioral constraints for CoC augmentation cover? Under what logical model should their conditions and action restrictions be checked for consistency? How should the checked specification be conveyed as natural language for learning? Free-form language is useful for human understanding, but making the semantics and assumptions required for automated verification explicit calls for additional structure. We therefore propose a framework connecting constraint explicitness, traceability, and logical consistency with natural-language learning inputs.

GuardSynth defines the scope, properties, and elements of behavioral constraints and specifies the subject and target, applicability, prohibited actions, persistence and release conditions, timing, and evidence using Evidence-Bound Lifecycle Contracts (EBLC). It lowers a selected profile to Core/SMT, verifies consistency and designated properties within a restricted model, and generates Controlled Natural Language (CNL) corresponding to specification fields for CoC augmentation. The formal specification serves as an intermediate representation for structuring and checking constraints, rather than replacing natural language as the final learning input. Specification verification and the behavioral effect of constraint augmentation are evaluated as logical and empirical evidence, respectively.

We evaluate Qwen3-VL-2B-Instruct with matched LoRA budgets within each task family and three optimization seeds across six tasks: maximum speed, minimum clearance, minimum entry delay, temporal entry after clearance, stopping, and lane changing. P0 is CoC alone, P1 adds our existing natural-language constraints, and P2 adds EBLC-derived CNL; P1 and P2 receive constraints during both training and evaluation. Both outperform P0 on the primary accuracy and violation measures. For temporal entry, mean accuracy is 49.86%, 96.81%, and 98.06%, respectively, and violation rates are 19.17%, 1.39%, and 0.83%. Separate specification-quality checks examine supported errors and boundary judgments. This separates the behavioral value of supplied constraints from the checkability added by the formal development path.

The paper makes three contributions:

- 1) A source-to-field procedure specifying which behav-

ioral constraints to select from CoC context, scene evidence, and supplied rules, while separating observations, rules, and modeling assumptions.

- 2) An EBLC-based development path with explicit profile semantics, restricted logical verification, field-to-CNL mappings, and defined insertion into model-facing CoC.
- 3) A task-to-verdict evaluation linking six behavioral tasks to implemented fields, formulas, generated sentences, and independent candidate judgments, alongside specification-quality checks.

Section II reviews related work, and Section III introduces background concepts in CoC/VLMs, logical verification, CNL, and LoRA. Section IV presents the proposed behavioral constraint model and GuardSynth’s specification, verification, and CNL generation method. Sections V and VI describe the experimental setup and results. Sections VII and VIII discuss the interpretation and scope of the findings and conclude the paper.

II. RELATED WORK

A. REASONING-BASED AUTONOMOUS DRIVING AND COC

Connecting visual observations to language-based reasoning and actions is an important direction in autonomous driving. Alpamayo-R1 combines Chain of Causation (CoC) reasoning with trajectory prediction and uses supervised fine-tuning and reinforcement learning to improve reasoning–action alignment. DriveLM represents relationships among perception, prediction, and planning questions and answers as a graph, while DriveVLM connects scene description, scene analysis, and hierarchical planning. These studies provide learnable representations of the process from observation to decision. [1], [2], [3]

Alignment between explanations and actions is also important for useful reasoning. Drive-R1 addresses reasoning–planning misalignment and history-based shortcut learning, using trajectory and meta-action rewards after supervised fine-tuning. Complementing this direction, we focus on how behavioral constraints are represented in learning data. We distinguish CoC explanations of why an action is taken from constraints restricting which actions are admissible in the situation, and combine the two. This distinction defines explicit normative information to accompany explanations, rather than presuming a general performance deficiency of existing reasoning models. [5]

B. TRAFFIC-RULE SPECIFICATION AND REGULATION-AWARE DECISIONS

Machine-interpretable traffic rules provide a direct foundation for formalizing behavioral constraints. Esterle et al. express traffic rules in linear temporal logic (LTL) and apply them to compliance assessment on driving data. This approach gives applicability conditions and temporal relationships explicit semantics. Following this principle, we structure the subject and target, activation, persistence, release,

and evidence links of CoC-augmenting constraints as EBLC elements. [6]

Retrieval-augmented reasoning offers another way to connect regulations with scenes. DriveReg retrieves situation-relevant traffic regulations and connects them to language-model reasoning to support compliance and interpretable decisions. Its central focus is retrieving and using applicable regulations. GuardSynth instead focuses on expressing selected behavioral constraints in an explicit intermediate representation, checking them within a restricted logical model, and conveying them as natural-language learning inputs. Regulation retrieval and constraint specification have complementary roles in obtaining supporting evidence and representing constraints. [7]

C. NATURAL LANGUAGE, FORMAL SPECIFICATIONS, AND CNL

Translating natural-language requirements into formal specifications connects human-readable descriptions with machine-verifiable representations. NL2TL uses abstracted atomic propositions and language models to translate natural language into temporal logic, separating domain-specific proposition recognition from translation structure. nl2spec associates language fragments with subformulas and lets users revise these associations to resolve ambiguity. The former emphasizes learning and adapting a translation model, while the latter emphasizes interactive specification and user correction. [8], [9]

Controlled Natural Language (CNL) is also relevant when specifications are presented in readable language. Attempto Controlled English (ACE) restricts grammar and interpretation rules to connect natural-language sentences with formal representations. GuardSynth's CNL is not an implementation of ACE; it maps fields and clauses of a selected EBLC profile to restricted sentence patterns. Our focus is traceable correspondence between defined constraint elements and generated sentences, rather than semantic equivalence across unrestricted natural language. Logical verification, sentence correspondence, and the validity of scene evidence are consequently distinct review targets. [10]

D. POSITIONING OF THIS WORK

Our preceding Safety-Constrained CoC manuscript already introduces natural-language execution guards and contract-pair evaluation over static, temporal, and maneuver tasks. GuardSynth retains that learning interface but develops a traceable source-to-specification-to-CNL path for producing its constraint component. The present P1 condition instantiates our earlier approach; P2 is its formally supported development path. Reused task generators and scoring principles are distinguished from newly executed experiments. [4]

These research directions establish foundations for learning driving reasoning, formalizing traffic rules, and connecting language with logic. Building on them, GuardSynth defines the scope, properties, and elements of CoC behavioral constraints and connects EBLC specification, restricted log-

ical verification, CNL generation, and CoC augmentation. The focus is on organizing learning constraints through a consistent representation and verification procedure, rather than developing a new driving model or a general-purpose translator. Table 1 compares the central objectives of the selected studies; it is neither an exhaustive feature audit nor a performance ranking.

We assess learning utility by comparing CoC alone (P0) with constraint-augmented CoC (P1 and P2), using the same model and matched training budgets. P1 is our existing natural-language constraint approach, and P2 uses CNL generated from EBLC. Thus, P1/P2 versus P0 is the primary comparison, while P1 versus P2 compares representations within our proposed family. This experiment evaluates the effect of supplying constraints; it does not isolate a causal effect of logical verification. The paper covers constraint specification, verification, CNL generation, and learning use. EBLC-based test generation and runtime monitoring are subsequent applications.

III. BACKGROUND

This section introduces established concepts needed to understand visual action reasoning and logical specifications: vision-language models and Chain of Causation, satisfiability-based verification, controlled natural language, and low-rank adaptation.

A. VISION-LANGUAGE MODELS AND CHAIN OF CAUSATION

Vision-language models (VLMs) process visual and textual information to generate descriptions or responses about a scene. Vision-language-action models (VLAs) connect such processing to action prediction. Alpamayo-R1 combines vision-language reasoning with trajectory generation for driving. [1]

Visual input, linguistic explanation, and action output serve different roles. The input provides observations; an explanation relates observations to a decision; the output represents the selected action. For example, answering "What is visible?" and "Which action should be selected next?" defines different learning targets. An action output can itself be a linguistic category such as "decelerate," rather than a trajectory or a steering and acceleration command. Descriptions of action prediction should therefore specify both the available inputs and the representation being learned.

Chain of Causation (CoC) expresses causally connected observations, rationales, and actions relevant to a decision. Alpamayo-R1 aligns these explanations with driving behavior. [1] An illustrative explanation is "the leading vehicle decelerates $\rightarrow$ the gap may shrink $\rightarrow$ decelerate to maintain separation." This example describes an action rationale; it is distinct from statistically identifying causal relationships from observational data.

The first part of this example is an observation, the second an anticipated consequence, and the last an action rationale. These arrows indicate explanatory flow rather than formal

TABLE 1. Central objectives of related work and their relationship to GuardSynth.

Study	Central objective	Representation or approach	Connection to this work
Alpamayo-R1; Drive-R1 [1], [5]	Reasoning–action alignment	CoC/CoT, supervised learning and RL	Useful supervision for action selection
DriveLM; DriveVLM [2], [3]	Reasoning from scenes to plans	Graph question answering; hierarchical planning	Structuring scene and action explanations
Esterle et al. [6]	Machine-interpretable traffic rules	LTl specifications	Explicit conditional and temporal semantics
DriveReg [7]	Regulation-aware decisions	Regulation retrieval and LLM reasoning	Connecting applicable rules and evidence
NL2TL; nl2spec [8], [9]	Formalizing language requirements	Temporal-logic translation; interactive revision	Language–specification correspondence
ACE [10]	Unambiguously interpreted language	Restricted grammar and formal interpretation	Principles of controlled expression
GuardSynth	Structuring constraints for CoC learning	EBLC $\rightarrow$ logical checks $\rightarrow$ CNL $\rightarrow$ CoC augmentation	Linking constraint model, verification, and learning inputs

implication. Learning from such explanations encourages a model to express these relationships. Whether the stated facts are supported by observations and whether the model selects an appropriate action remain separate questions.

B. LOGICAL SPECIFICATIONS AND SATISFIABILITY-BASED VERIFICATION

SAT determines propositional satisfiability, while SMT also considers background theories such as arithmetic. [11] Z3 is a solver for such SMT queries. [12]

1) Satisfiability and Counterexamples

Let Φ combine a specification and its assumptions, and let P denote a property. Satisfiability of Φ establishes the existence of a model satisfying the combined conditions. Property verification can be expressed by checking the counterexample formula:

$$\Phi \wedge \neg P$$

A satisfying model violates the property; unsatisfiability establishes that Φ entails P . Because inconsistent assumptions entail any property vacuously, satisfiability of Φ must also be checked. When temporal states are encoded over finitely many indexed steps, the conclusion applies to that bound and those assumptions. Its scope is determined by the facts and relationships represented in the specification. [11]

Consider an elementary arithmetic example: x is an integer and Φ is ' $0 \leq x \leq 5$.' The assignment $x = 3$ satisfies Φ . Here, a model means an interpretation of logical variables and symbols that satisfies the formula, not a trained neural network. If P is ' $x \leq 10$,' the counterexample query asks whether ' $0 \leq x \leq 5$ and $x > 10$ ' can hold. No integer satisfies it, so the query is UNSAT and $\Phi \models P$ holds.

If instead P is ' $x \leq 2$,' assigning $x = 3$ satisfies Φ but violates P . The counterexample query is SAT, and this assignment refutes the claim that P always holds. SAT for a counterexample query does not mean the original specification is

inconsistent: it identifies a case permitted by the specification that violates the property. SAT and UNSAT must therefore be interpreted with respect to the particular query.

2) Finite Temporal Bounds and Assumptions

Temporal change can be represented by indexed states and relations between adjacent states. For example, s_0 , s_1 , and s_2 denote three decision states with two intervening transitions. Unless a time interval is separately defined, these indices do not denote physical seconds. A bounded verification problem combines initial conditions, transition relations, and properties within the selected range.

This distinction matters when interpreting a result. The absence of a counterexample over three states does not establish a property at every future time. Likewise, when observed facts are supplied as assumptions, logical verification addresses what follows from those assumptions rather than independently establishing their observational accuracy. Specification consistency, property satisfaction, and the correctness of input facts are distinct questions.

C. CONTROLLED NATURAL LANGUAGE

Controlled natural language (CNL) deliberately restricts the vocabulary, grammar, or interpretation of a natural language. It supports precision and consistency while retaining natural-language readability. CNLs intended to improve document comprehension differ in purpose and rigor from those designed to correspond to formal representations. [13]

For formally oriented CNL, interpretation rules for conditions, negation scope, and logical relationships are important. Precision depends on explicit definitions of permitted expressions and their interpretation, rather than fluency alone. Describing a CNL therefore requires attention to its particular grammatical and semantic rules. [13]

As a simple illustration, let p mean "the device is locked" and q mean "the device operates." The sentence "If the device is locked, it does not operate" can be assigned the interpretation $p \rightarrow \neg q$. It restricts operation while locked, but does not

require operation whenever the device is unlocked. Rewriting it as “If the device is unlocked, it operates” instead gives $\neg p \rightarrow q$, a different condition. Natural phrasing and logical equivalence are separate issues.

In this example, p and q fix the meanings of terms, “if” expresses the conditional relationship, and “not” determines the scope of negation. An explicit correspondence makes the condition and conclusion identifiable. Preserving meaning between language and formulas concerns these lexical, grammatical, and logical relationships, not merely similar words or sentence lengths. This example illustrates a CNL principle without defining a domain-specific constraint language.

D. LOW-RANK ADAPTATION

Supervised fine-tuning adapts a pretrained model using examples of inputs and target outputs. For language outputs, training can encourage generation of target tokens given an input. Full fine-tuning updates all model weights selected for training, whereas parameter-efficient fine-tuning learns a subset of parameters or a small additional parameter set.

Low-Rank Adaptation (LoRA) fine-tunes a model by freezing pretrained weights and learning an update represented by low-rank matrices. For a pretrained weight matrix W_0 , the adapted weight is expressed as follows. [14]

$$W = W_0 + \frac{\alpha}{r}BA$$

Here, W_0 is the frozen pretrained weight and W is the effective adapted weight. For a linear map with input dimension k and output dimension d , both have size $d \times k$. The trainable matrices A and B have sizes $r \times k$ and $d \times r$, respectively, where r is a positive internal dimension usually smaller than d and k . Their product BA therefore has the same shape as W_0 and rank at most r . The scalar setting α scales the update through α/r . [14]

Only A and B are learned rather than the entire W_0 . With sufficiently small r , the trainable parameter count for that matrix decreases from dk to $r(d + k)$. The choice of r controls both the space of representable updates and the parameter count, so its role differs from a simple scaling coefficient.

For an illustrative calculation with $d = k = 512$ and $r = 8$, the full matrix contains 262,144 values, whereas A and B together contain 8,192 trainable values, or 3.125% as many. This is a one-matrix example, not a whole-model memory reduction or an experimental setting. LoRA specifies how adaptation is performed; the accuracy and constraint compliance of learned actions require separate evaluation.

IV. PROPOSED METHOD

A. OVERVIEW AND INPUTS AND OUTPUTS

GuardSynth preserves the action explanation in CoC while making conditions that restrict action selection explicit in a separate specification, then returning those conditions to a natural-language representation. Inputs comprise original CoC, scene observations available for the decision, and applicable rules with their evidence. Outputs comprise structured

behavioral constraints, logical verification results under a restricted model, controlled natural language (CNL) generated from the specification, and an augmented learning input. The formal representation specifies condition semantics and verification scope; the language representation conveys those conditions to learning.

The process first selects candidate constraints and checks their evidence, then specifies accepted constraints in EBLC. Structural and semantic checks precede conversion to a verification-oriented intermediate representation and SMT queries. CNL corresponding to the specification is subsequently generated and combined with original CoC.

The inputs in Figure 1 provide non-interchangeable information. Original CoC supplies action and rationale context for identifying relevant constraint candidates. The other three inputs are defined below as observational values, behavioral rules, and interpretation scope.

Observations: definition and example. An observation describes the state of an entity, zone, or event at a specified time. Real-scene observations are linked to evidence such as video and its confirmation, whereas the primary synthetic experiment uses states supplied by the generating environment. “The conflict zone became clear at 11.0 seconds” is an event-time observation. Stage 01 in Figure 1 binds the entity and event; stage 03 binds $\text{clear_ms} = 11000$ as an environmental query variable. This observation alone does not determine a required waiting duration.

Rules: definition and example. A rule specifies which action is restricted under an applicability condition and when that restriction is released. “Do not enter for at least 0.5 seconds after clearance” is a supplied experimental rule. Stage 01 in Figure 1 confirms its applicability evidence; stage 02 specifies the restricted action and $\text{release_clear_ms} = 500$. The duration is neither an observed scene property nor a value inferred from CoC.

Modeling assumptions: definition and example. A modeling assumption states the scope within which inputs are interpreted and verified. “The zone is not reoccupied after clearance” allows continuous clearance to be represented by one clearance time. Stage 02 in Figure 1 records this assumption as contract scope; stage 03 uses it in the logical interpretation. If reoccupation is possible, the assumption must be reconsidered before applying the same single-time expression.

Consequently, the same observation at 11.0 seconds yields admissibility from 11.5 or 12.5 seconds under supplied waits of 0.5 or 1.5 seconds. Stage 04 in Figure 1 conveys the specified and verified restriction through CNL. Separating these roles prevents treating an observation as a rule or an assumption as an observed fact.

Executable numerical and qualitative profiles instantiate this approach. The learning experiments use a temporal profile and static/maneuver candidate-summary profiles constructed from supplied synthetic rules. A separate qualitative profile connects entities, zones, and events through observations and human review of real scenes. Both link speci-

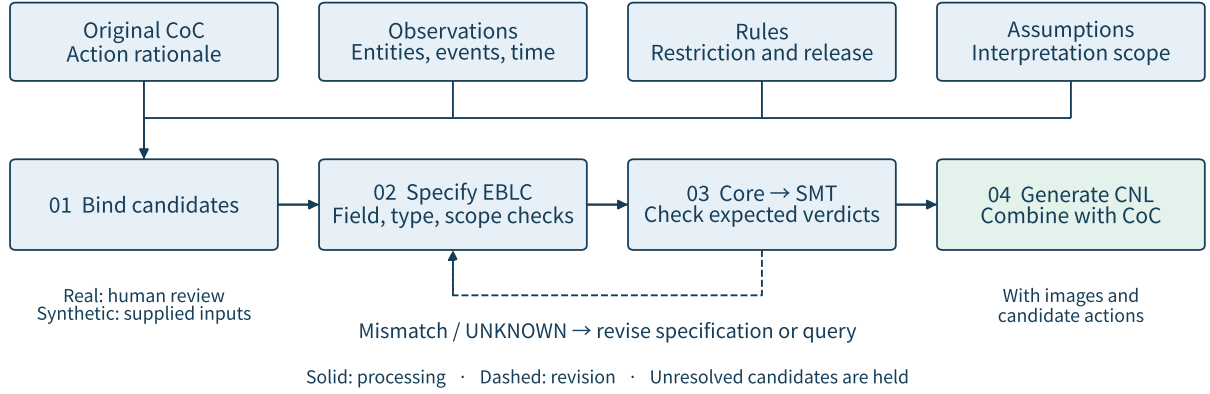


FIGURE 1. The upper row distinguishes four input roles; stages 01–04 below run from evidence binding to CoC augmentation. Solid arrows show processing order; the dashed path returns unexpected verdicts or UNKNOWN for revision. Unresolved candidates are held at stage 01. Real scenes require human confirmation of applicability evidence; synthetic experiments bind supplied conditions.

fifications to evidence, but their inputs and semantics differ. We use the temporal contract as the running example and the qualitative profile to describe evidence uncertainty and constraint lifecycles. Implementation evidence is linked in S1 and S2.

B. FORMAL SPECIFICATION WITH EBLC

1) Scope and Elements

A behavioral constraint restricts available actions in a situation. It differs from an observation, an action rationale, and an objective that ranks admissible actions. Satisfying an entry-permission condition does not force entry; selecting among permitted actions requires a separate objective.

Evidence-Bound Lifecycle Contracts (EBLC) represent constraints with their targets, conditions, action restrictions, persistence and release, and evidence. Evidence-Bound denotes explicit links to the inputs on which conditions and rules depend. Lifecycle denotes activation, persistence, release, and, where required, reactivation. This paper uses these organizing principles and implemented restricted profiles, rather than assuming a language covering every traffic situation.

The qualitative profile additionally represents event truth, evidence validity, initial activation, and a policy for combining obligations. These elements address uncertain scene evidence; they are not all evaluated in the primary temporal learning experiment. The proposed modeling scope and experimentally evaluated scope are distinguished.

2) Contract Definition and Interpretation

We use $C = (s, z, \Sigma, O, E, V)$ to explain the profile organization: s denotes the actor, z the target or zone, Σ the scope and modeling assumptions, O the set of restrictive obligations, E the observation and rule evidence links, and V the interpretation version. This is explanatory notation, not a new six-field JSON schema. Actual serialization follows the restricted profile in Code Box 1.

An obligation is interpreted through its activation condition, persistence and release rules, restricted action, and any required initial state. The temporal profile compresses these into a single clearance time and a waiting duration. The qualitative profile updates activation from evidence and the previous state at each decision step. Assumptions and state variables from different profiles are not interchangeable.

Let h_t denote the supplied evidence and state history through decision step t , and a a current action candidate. The meaning of a contract is its admissible candidate set $A_C(h_t)$. Membership means compatibility with the stated conditions and restrictions. Subscript t identifies the decision step, and C identifies the contract. Multiple obligations combine their admissible sets by intersection, requiring all applicable restrictions to hold. The qualitative profile also imposes an entry policy requiring current clearance evidence. Previous inactivity is therefore distinct from current entry permission.

An independent objective selects among admissible candidates. Deferral and entry after sufficient waiting may both be permitted while receiving different progress scores. Separating admissibility from optimization prevents interpreting compliance as optimality.

3) Temporal Syntax and Semantics

The temporal profile accepts a structured JSON contract. The following is actual generator output for a 0.5-second waiting rule. Source references provide traceability and contain no target action label.

Code Box 1. Actual generated temporal contract JSON

The input is a supplied 0.5-second waiting rule; the output is a contract containing its target, release condition, and assumptions. Line numbers refer to positions within this box.

TABLE 2. Constraint representation and example mappings

Element	Specification role	Temporal instantiation
Subject and target	Whose action is restricted, and with respect to what?	Ego and pedestrian conflict zone
Scope	Situation and model of interpretation	Single clearance transition without reoccupation
Action restriction	What is disallowed?	Entry before release conditions hold
Persistence and release	When does the restriction persist or end?	Required duration following clearance
Time and units	Instants, durations, and boundaries	Integer milliseconds; inclusive threshold
Evidence and version	Rule sources and interpretation	Rule references and profile version

```

1 {
2   "version":
3     "supplied-temporal-ebc-profile-v0.1",
4   "subject": "ego",
5   "target": "pedestrian_conflict_zone",
6   "hold": "NO_ENTRY_WHILE_OCCUPIED",
7   "release_clear_ms": 500,
8   "time_unit": "ms",
9   "source_kind": "SUPPLIED_SYNTHETIC_RULE",
10  "assumption": "SINGLE_CLEAR_TRANSITION_NO_
11    REOCCUPATION",
12  "source_refs": [
13    "Project01/temporal_guard_world.py::_guard",
14    "paper1-method-preservation-v0.1"
15  ]
16 }

```

The `subject` and `target` identify the actor and restricted zone. The `hold` field specifies no entry during occupancy; `release_clear_ms` specifies waiting after clearance. The `assumption` field specifies one occupied-to-clear transition without subsequent reoccupation. This permits continuous clearance to be represented by one clearance time and an elapsed duration.

In Code Box 1, lines 2–5 fix the version and restriction target; lines 6–7 specify duration and units. Lines 8–9 identify the supplied synthetic rule and no-reoccupation assumption, while lines 10–13 preserve source references. Environmental clearance and candidate entry times are bound separately rather than mixed into this rule JSON.

Evaluation separately binds environmental clearance time `clear_ms`, candidate entry time `entry_ms`, and Boolean entry indicator `enters`. Both times use integer milliseconds. The restriction is:

$$e \Rightarrow u_{ms} \geq \tau_{ms} + \Delta_{ms}.$$

For `clear_ms = 11000` and `release_clear_ms = 500`, an entering candidate is compatible only at or after 11500ms. The inclusive comparison admits 11500ms but excludes 11499ms. A non-entering candidate has `enters = false` and does not violate this entry restriction; its progress is assessed separately.

The validator checks the supported version and fixed field combinations. Supported durations are 500, 1500, 800, and

1800ms; the primary expanded learning dataset uses 500 and 1500ms. The implementation accepts a restricted profile with defined interpretation, not arbitrary JSON as a general contract. S1, S3

Writing $\tau = \text{clear_ms}$, $\Delta = \text{release_clear_ms}$, $u = \text{entry_ms}$, and $e = \text{enters}$ gives $P_C(e, u, \tau) = \neg e \vee (u \geq \tau + \Delta)$. Candidates satisfying P_C belong to the temporal admissible set. Both τ and u are instants, whereas Δ is a duration. This distinguishes a zone being clear from having remained clear for the required duration.

4) Qualitative Lifecycle Semantics

The real-scene profile uses `truth[j,t]`, `valid[j,t]`, and `active[j,t]` for obligation j at decision step t . Truth values are TRUE, FALSE, UNKNOWN, and CONFLICT. FALSE denotes confirmed resolution of the restricting condition, not mere absence of an observation. Validity denotes admissibility of the evidence. Activation records whether the obligation is active; its value before the first decision is a separate input.

Code Box 2. Pseudocode for evidence-based state updates

Inputs are the current truth value and evidence validity for obligation j , together with previous activation; the output is current activation. This is explanatory pseudocode, not a verbatim source excerpt.

```

1 on[j,t] = valid[j,t] AND (truth[j,t] = TRUE)
2 clear[j,t] = valid[j,t] AND (truth[j,t] = FALSE)
3 active[j,t] = TRUE, if on[j,t]
4   FALSE, else if clear[j,t]
5   previous_active, otherwise

```

Evidence is read and activation updated before constraining the action. Valid activation evidence activates an obligation, valid clearance evidence releases it, and otherwise its previous state persists. The same rule reactivates it when the activation condition is confirmed again. Explicit ordering and initialization fix the interpretation at each step.

The current policy permits entry only when clearance of every declared obligation is confirmed. Previous inactivity alone does not permit entry if current clearance is unconfirmed. When all obligations are clear, both entry and deferral are allowed. Permission is not a liveness guarantee requiring progress. UNKNOWN and CONFLICT are evidence states,

distinct from an SMT solver returning UNKNOWN when it cannot decide a query. S2

Lines 1–2 of Code Box 2 require both validity and the corresponding truth value. Lines 3–5 select activation, release, or persistence. At the first step, `previous_active` is the explicit `prior_active` input; subsequently it is activation at the preceding step. A single truth value cannot be both TRUE and FALSE, so activation and clearance cannot simultaneously hold in this encoding.

The policy is `entry_permitted[t] = AND_j clear[j,t]`; ENTER_ZONE requires this value to be true. DEFER_ENTRY does not violate this entry restriction. Unknown, conflicting, or invalid evidence separately sets `review_required`. For example, starting from inactivity, valid evidence `TRUE → UNKNOWN → FALSE → TRUE` yields `active → active → inactive → active`. Entry is permitted only at the third decision, where deferral remains possible. This is a constructed semantic illustration.

5) Why Use an EBLC Profile?

JSON specifies a serialization structure; by itself it does not assign lifecycle semantics, define the interpretation of unknown evidence, or establish what a generated sentence means. General temporal logics can express many of these conditions, and hand-written rules can implement them. EBLC is not claimed to be more expressive than those alternatives. Its role here is to fix a domain-specific contract between evidence fields, interpretation rules, supported Core lowering, and model-facing language. For example, `release_clear_ms` denotes a duration rather than an event time, a `ge` bound is inclusive, and `source_refs` records provenance rather than a model answer.

The contribution is thus an explicit development interface with checkable mappings, not the invention of implication, JSON, or SMT. Each implemented profile has a validator, logical interpretation, and renderer; unsupported combinations are rejected. The experiments evaluate the utility and checks of this path, not superiority over an equally specified JSON-to-SMT system. A bare schema would become an equivalent engineering alternative only after supplying the corresponding semantics, lowering, provenance policy, and language mapping.

C. DEFINING AND DERIVING BEHAVIORAL CONSTRAINTS

1) Separating Candidate Constraints from Evidence

Not every CoC sentence becomes a constraint. “A pedestrian is crossing” is an observational claim; “yield to protect the pedestrian” is an action rationale. “Do not enter while the zone is occupied” contains a condition and restriction, making it a constraint candidate. “Choose the admissible candidate with greatest progress” is a selection objective. This distinction avoids inventing thresholds or release conditions absent from the explanation and its evidence.

Inputs serve three evidence roles. CoC identifies potentially relevant actions and obligations. Scene observations

bind entities, zones, events, and times. Supplied rules or separately established normative evidence determine applicability and restriction content. A sentence appearing in CoC does not itself confirm applicability to the current scene.

2) Specification Procedure and Automation Scope

The procedure identifies action-relevant entities, situations, and rationales in CoC, then binds entities and zones to observations and identifies the decision time. Rule evidence determines the restricted action and applicability conditions; persistence, release, timing, and assumptions are subsequently specified. Unconfirmed information remains unresolved rather than becoming an observed fact. Accepted content is mapped to supported fields with evidence references and version information.

In the primary synthetic experiment, the environment generator and supplied rule provide these inputs. Environmental clearance and rule duration determine the contract and verification representation. Automation structures and transforms given conditions; it does not discover traffic law or waiting durations from video alone. The real-scene path involves human confirmation of entities, zones, events, and applicability, and the implemented adapter is limited to a particular review task. S3, S4

Constraints and learning references are managed separately. Defining admissibility does not automatically establish a real-scene ground-truth action. For the synthetic task, separate integer arithmetic uses known environmental states and rules to identify admissible candidates, then selects the one with greatest progress. This calculation does not copy its answer from CNL or an SMT verdict, but shares the environmental and rule assumptions.

3) Running Temporal Example

Original CoC reads “Yield to the crossing pedestrian, then proceed through the intersection.” The environment supplies clearance at 11.0 seconds; a separate rule requires a minimum wait of 0.5 or 1.5 seconds. The duration is an explicit experimental condition, not a value inferred from CoC or a universal traffic-law requirement.

Progress is a synthetic utility, not measured distance or speed. B maximizes admissible progress under the 0.5-second contract and C under the 1.5-second contract. D violates neither but does not complete the progress objective. The table is a generated data instance, not evidence of actual model predictions. Candidate labels rotate during dataset construction to avoid linking the answer to a fixed letter.

4) What Is Extracted, From Where, and Where It Is Placed

Constraint extraction here means selecting and binding action-relevant content to a supported specification, not converting every sentence into a rule. The following source allocation prevents a model or annotator from inventing a normative threshold from an image.

In the numerical experiments, bounds are read from supplied rule fields (`threshold`, `max_decel`, `max_jerk`, `min_gap`,

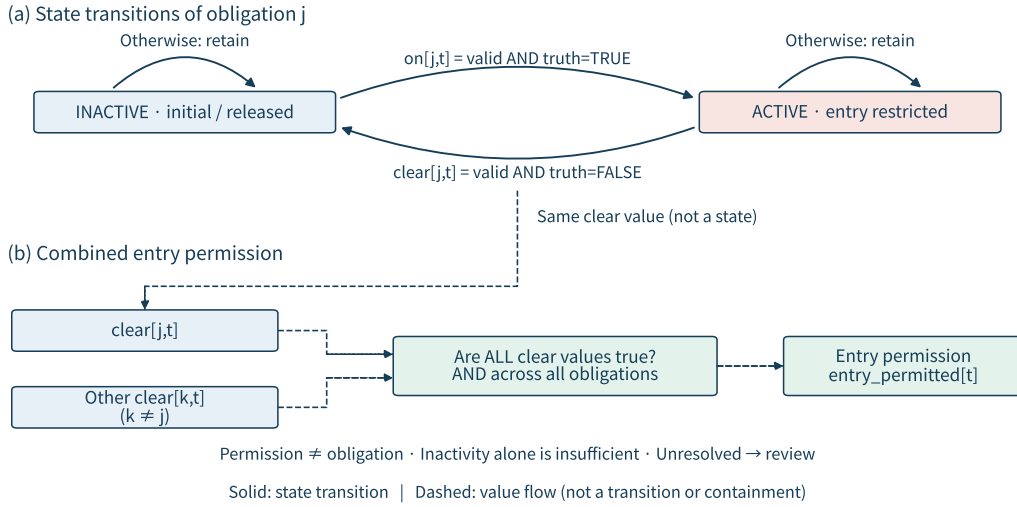


FIGURE 2. (a) shows the state transitions of an individual obligation j ; (b) combines all obligations' clearance conditions to determine entry permission. Solid arrows denote transitions or state retention; dashed arrows carry current clear values. The release condition in (a) and its corresponding input in (b) use the same evidence-derived value, combined with other obligations' clear values by AND. Panel (b) is neither a third state nor a nested state of (a). Because inactivity includes both initial inactivity and release, permission depends on current clearance of every obligation, not inactivity alone.

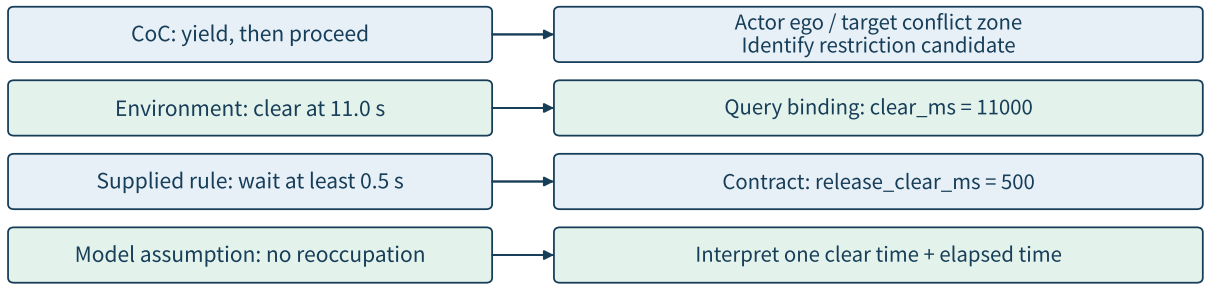


FIGURE 3. Environmental clearance at 11.0 seconds binds a query variable, whereas the supplied 0.5-second duration becomes a contract field. No reoccupation is an assumption needed to connect these values through a temporal inequality. Derivation distinguishes these roles when constructing evidence-supported fields.

or the temporal wait) rather than inferred from pixels. The static and maneuver compiler stores each bound as `field/op/unit/ticks` under `limits`; `scale=1000` expresses integer milli-units after checking the external unit. Candidate measurements are bound only when a candidate is queried. Real-scene development instead requires reviewer-confirmed observations and applicability before specification; extraction accuracy is not established by the synthetic learning scores.

The output sentence is appended to the existing rationale in the model-facing Requirement CoC text, before candidate descriptions and the selection instruction. The actual static data use `rich_coc_text = COC_TEXT + " " + CNL`; maneuver data append `constraint_text` to the CoC requirement. The separate record retains EBLC, provenance, verification results, and the reference label. Neither the reference label nor SAT/UNSAT is inserted as a hint. This defines both the content to extract and its precise destination.

D. VERIFYING EBLC SPECIFICATIONS LINKED TO COC

1) Binding Checks and Core Lowering

Verification requires semantic input binding and logical queries. Subject, zone, event time, and duration must refer to the same case, and specifications must satisfy supported version, field, and unit requirements. Evidence identifiers support traceability rather than proving factual truth. Unbound fields are distinguished from completed scene bindings; passing syntax checks does not turn a conditional specification into reviewed learning data.

Temporal contracts lower to Core declarations of Boolean `enters` and integer `clear_ms` and `entry_ms`; `release_clear_ms` becomes an integer constant. Core is an intermediate representation of declarations and logical clauses, type-checked before SMT compilation. Time uses integer millisecond ticks with Core unit marker 1, not seconds. Variables in this profile are time-invariant and the clause is enforced initially. The configured `horizon = 2` is a model bound, not verification

TABLE 3. Constraint representation and example mappings

Candidate	Entry time	Progress score	0.5-second wait	1.5-second wait
A	10.5s	110	Violates	Violates
B	11.5s	100	Admissible; reference	Violates
C	12.5s	85	Admissible	Admissible; reference
D	No entry	0	Admissible	Admissible

TABLE 4. Constraint representation and example mappings

Source	Selected content	Destination
Original CoC	Intended action, rationale, potential obligation	Constraint kind and relevance candidate; preserve the original CoC separately
Scene evidence or synthetic state	Target, zone, decision time, clearance event, candidate measurements	Entity bindings and query assignments; not rule thresholds
Supplied rule or reviewed rule evidence	Restricted action, applicability, comparison, bound, release requirement	EBLC obligation or <code>limits</code> fields with source references
Explicit modeling decision	Time scope, no reoccupation, completion convention	Versioned profile assumptions, disclosed with evaluation scope

of a two-second driving trajectory. S1

2) Properties and Queries

We distinguish consistency, restriction enforcement, release boundaries, and permissibility. Base SAT establishes a satisfying model, but non-entry alone can satisfy the specification. We therefore separately check that an entering model exists after release. This avoids specifications that exclude all entry without substituting for measurement of actual model progress.

Concrete times assume clearance at 11.0 seconds. The restriction query searches for a counterexample with `enters` true and `entry_ms < clear_ms + release_clear_ms`. This is symbolic rather than fixing both time variables; boundary rows bind concrete values. Even the symbolic result concerns the encoded inequality and assumptions, not video interpretation or rule selection.

The six manuscript queries were executed for both 500ms and 1500ms contracts, yielding 12 matching verdicts. The existing bridge's 100 queries were rerun separately. Neither set constitutes new learning-effect evidence. Records preserve each query, expected and observed verdicts, and actual SMT-LIB output. S5

3) SMT Representation and Interpretation

The following readable SMT-LIB example expresses the core 0.5-second restriction. It is an explanatory representation, not the entire compiler output, which is retained in separate query files.

Code Box 3. Executable SMT-LIB example comparing early and boundary entry

Inputs are the contract duration and clearance at 11000ms. Each candidate time is fixed in turn to query compatibility with the contract.

```

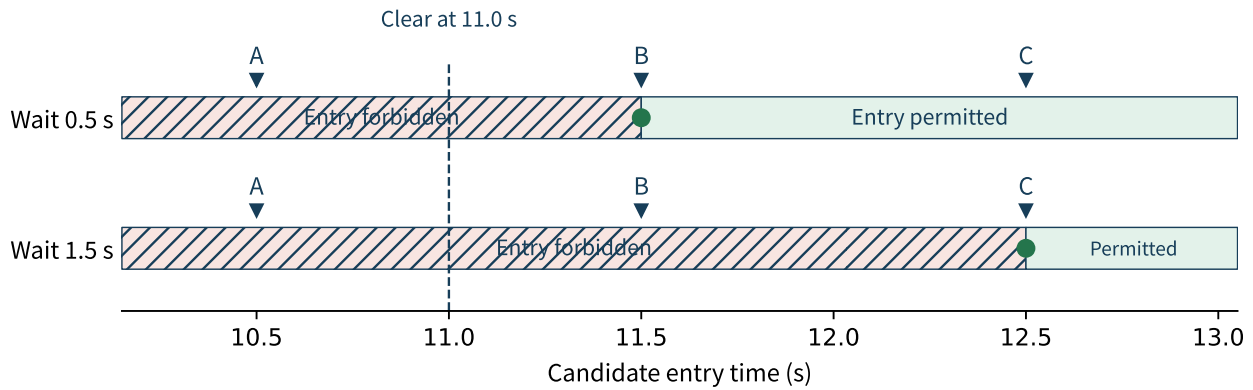
1 (declare-const enters Bool)
2 (declare-const entry_ms Int)
3 (declare-const clear_ms Int)
4 (assert (= clear_ms 11000))
5 (assert (=> enters (>= entry_ms (+ clear_ms
500))))
6 (assert enters)
7 (push)
8 (assert (= entry_ms 11499))
9 (check-sat) ; unsat
10 (pop)
11 (push)
12 (assert (= entry_ms 11500))
13 (check-sat) ; sat
14 (pop)

```

The first UNSAT verdict indicates incompatibility of early entry with the specification, not a defective specification. The second SAT verdict indicates compatibility of the selected entry. Neither includes optimality or real-world safety. UNKNOWN or unexpected verdicts are not accepted as successful verification. Structural checks, scene binding, logical verification, and CNL correspondence serve different roles.

Lines 1–3 of Code Box 3 declare types; line 4 binds environmental clearance and line 5 states the restriction. Line 6 excludes satisfying the query through non-entry. Lines 7–10 temporarily add 11499ms, obtain UNSAT, and remove that candidate condition. Lines 11–14 add 11500ms to the same base contract and obtain SAT. The push/pop scopes prevent both candidate conditions from remaining simultaneously and causing an artificial contradiction.

E. CNL GENERATION AND COC AUGMENTATION



Constructed contract example · D does not enter · Not a performance plot

FIGURE 4. The horizontal axis is entry time in seconds. The same clearance event yields admissibility from 11.5 or 12.5 seconds depending on the duration. Closed boundary points include the threshold instant. D is a non-entry candidate, not a point on this axis. The graph computes contract semantics rather than model accuracy or measured driving performance.

TABLE 5. Constraint representation and example mappings

Property	Query	Expected	Interpretation
Consistency	Base contract	SAT	A satisfying case exists
Restriction	Entry earlier than release	UNSAT	Early entry and contract are incompatible
Before boundary	Entry at 11.499s; wait 0.5s	UNSAT	Entry just before release is excluded
At boundary	Entry at 11.500s; wait 0.5s	SAT	Equality is permitted
After boundary	Entry at 11.501s; wait 0.5s	SAT	Entry after release is feasible
Deferral	enters is false	SAT	Entry is not forced

1) Generating Sentences from Verified Specifications

CNL is generated from checked fields through fixed sentence rules. The temporal renderer accepts structurally validated contracts and maps zone and duration to text. It does not itself invoke the solver; specification verification precedes rendering. Candidate labels, reference actions, and progress scores are not generator arguments. S1

Code Box 4. Verbatim excerpt of the CNL generator

The following is the actual render function in S1. Input raw is a temporal contract; the return value contains two sentences. Numbers count logical source lines; long lines wrap within the box.

```

1 def render(raw):
2     validate(raw)
3     # Render only validated contract fields, never
4     # scene target/candidate/answer.
5     return ('Do not enter the pedestrian conflict
        zone while it is occupied. '
        f'Enter only after the zone has remained
        clear for at least {raw["release_clear_
        ms"]/1000:.1f} s.')
```

Line 2 requires an exact match to a supported contract. Since validation also fixes the target and restriction type, line 4 can emit a constant zone phrase. Line 5 converts milliseconds to seconds with one decimal place, exactly represent-

ing the currently supported durations. The function neither invokes the solver nor selects a target action. Other targets or arbitrary precision would require a separate extension.

Actual output for the 0.5-second contract is:

Do not enter the pedestrian conflict zone while it is occupied. Enter only after the zone has remained clear for at least 0.5 s.

The first sentence identifies the zone and occupancy restriction; the second expresses release and duration. “At least” corresponds to an inclusive boundary, and “only after” specifies a necessary condition rather than an instruction to enter. The conversion maps 500ms to 0.5s and, under the same rule, 1500ms to 1.5s. Correspondence is interpreted for supported durations and the single-clearance, no-reoccupation assumption.

2) Scope of Semantic Correspondence

Generation preserves target, negation, conditional direction, value, and unit. Omitting “at least” or replacing “enter only after” with a requirement to enter after clearance would change admissibility. Fixed field mappings make these relationships explicit and inspectable.

Not all metadata appears in model-facing sentences. Versions and source references remain traceability metadata;

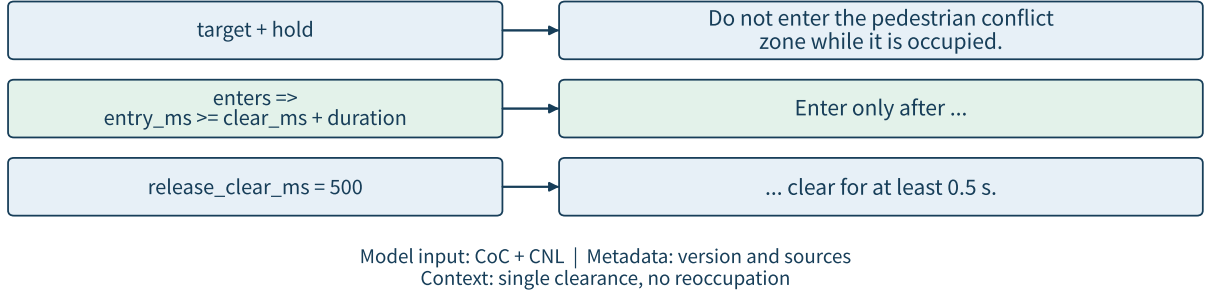


FIGURE 5. Target and restriction map to the first sentence; release duration and the inclusive boundary map to the second. Original CoC and CNL enter the model together, while source and version metadata remain separately traceable. Colors distinguish mappings, not additional levels of verification.

CNL conveys action-relevant conditions. Modeling assumptions are specified in the environment and task definition. The claim is correspondence of action restrictions within that context, not literal one-to-one rendering of every JSON field. Freely rewritten sentences or additional profiles require separate semantic review.

3) Combining CNL with Learning Inputs

Original CoC provides an action rationale, and CNL supplies additional behavioral conditions. They are combined without replacing CoC with a specification. In the comparison below, the resulting text is **original CoC + added CNL**; the right column shows only the added component. Temporal rows reproduce synthetic training text, whereas the STOP row summarizes and translates the core of a real-scene development output.

The first two rows pair different waiting rules for the same task, not different scene categories. If the zone clears at 6.0 s, entry at 6.5 s meets the 0.5-second condition but not the 1.5-second condition. CNL thus distinguishes prohibition and release conditions absent from the shared CoC; the durations are supplied experimental rules. The STOP row expresses a prohibition common to moving and stationary states and is separate from the main synthetic performance experiment. Constraint augmentation also does not automatically establish the factual accuracy of the original CoC: in development scene #68, the original red-light claim remained unconfirmed, while the constraint was grounded in a separately confirmed worker’s STOP instruction.

The image and action candidates accompany this text, while a separate reference calculation supplies the target label. P0 receives original CoC, P1 receives CoC with the existing natural-language constraint, and P2 receives CoC with EBLC-derived CNL. P1 and P2 both add constraints; their primary comparator is CoC-only P0. Raw EBLC JSON and solver verdicts are not supplied as answer hints.

P1 and P2 receive constraints during both training and evaluation. This evaluates selection using explicit rules, not directly whether identical behavior persists without evaluation-time constraints. P0 lacks the waiting duration, so its inputs can be identical for paired contracts requiring different refer-

ence actions. This information difference is disclosed as part of the comparison.

Traceable records retain image references, CoC, contracts and evidence, CNL, candidates, reference actions, and splits, separating model-visible fields from answers and verification fields. This supports reconstruction of what was specified from which evidence, how it was expressed, and which action served as the learning target. Training budgets and metrics are presented in the experimental section.

F. FROM TASK TO CONSTRAINT FIELDS, FORMULA, CNL, AND VERDICT

Table 7 links the six evaluated tasks to actual supported contract fields and logical clauses. The examples are drawn from frozen test contracts: static scene 20000, stopping scene 20000, and lane-change scene 20001; the temporal row uses the running boundary query. They illustrate the implementation, not six independently collected road videos.

Let q denote *completes*, e denote *enters*, and $\hat{x}$ denote a candidate measurement scaled by 1000. Each non-temporal Core clause is $q \Rightarrow (\hat{x} \text{ op } b)$, where b is the stored integer *ticks*. For stopping, three clauses are conjoined. Static candidates all complete; noncompletion in a maneuver is nonviolating but does not count as goal completion. The generated CNL conveys numeric bounds but does not separately render the Core exemption for noncompletion. Correspondence of bounds for goal-completing candidates is therefore distinguished from whole-contract equivalence over noncompleting behavior; the latter is not established here. The profile checks stored candidate summaries, not continuous trajectories.

For stopping, the two remaining generated clauses are “The candidate’s peak deceleration must be at most 5.8 m/s^2 .” and “The candidate’s peak jerk must be at most 7.4 m/s^3 .” Thus the table abbreviates the CNL display, not the checked contract. Here o is signed stop offset, a peak deceleration, j peak jerk, g minimum lane-change gap, v maximum speed, d minimum obstacle clearance, and u entry delay. Temporal clearance differs from a fixed delay from now: it binds an observed clearance instant before adding the required wait.

TABLE 6. Constraint representation and example mappings

Example	Original CoC	Added CNL
Pedestrian yielding, 0.5-second rule	Yield to the crossing pedestrian, then proceed through the intersection.	Do not enter the pedestrian conflict zone while it is occupied. Enter only after the zone has remained clear for at least 0.5 s.
Pedestrian yielding, 1.5-second rule	Yield to the crossing pedestrian, then proceed through the intersection.	Do not enter the pedestrian conflict zone while it is occupied. Enter only after the zone has remained clear for at least 1.5 s.
Worker's STOP sign, development example	Stop at the stop sign held by the construction worker for the one-way alternating traffic zone ahead.	While the confirmed stop instruction applies to ego at the current decision, do not start or accelerate even if motion state is unknown. This does not prohibit deceleration intended to stop or continued waiting; instruction clearance is not established.

TABLE 7. Executed task-to-verdict mappings; CNL cells quote generated clauses

Task	Constraint fields	Actual logical condition	Generated CNL	Candidate judgment
Maximum speed	speed/le/m/s/4400	$q \Rightarrow \hat{v} \leq 4400$	The candidate's maximum speed must be at most 4.4 m/s.	A: 6.0 m/s, UNSAT; B: 2.8 m/s, SAT
Minimum clearance	clearance/ge/m/2000	$q \Rightarrow \hat{d} \geq 2000$	The candidate's minimum obstacle clearance must be at least 2 m.	A: 0.9 m, UNSAT; B: 3.1 m, SAT
Minimum entry delay	entry_delay/ge/s/2500	$q \Rightarrow \hat{u} \geq 2500$	The candidate's intersection entry time from now must be at least 2.5 s.	A: 0.9 s, UNSAT; B: 4.1 s, SAT
Temporal clearance	release_clear_ms=500; clear_ms=11000	$e \Rightarrow u_{ms} \geq 11000 + 500$	Enter only after the zone has remained clear for at least 0.5 s.	11499 ms: UNSAT; 11500 ms: SAT
Stopping	stop_offset_m, peak_decel, peak_jerk; bounds 0, 5800, 7400	$q \Rightarrow (\hat{o} \leq 0 \wedge \hat{a} \leq 5800 \wedge \hat{j} \leq 7400)$	Stop at or before the stop line (stop offset at most 0 m).	C beyond line: UNSAT; D within all bounds: SAT and preferred
Lane change	min_gap_m/ge/m/5800	$q \Rightarrow \hat{g} \geq 5800$	The candidate's minimum gap during the lane change must be at least 5.8 m.	D: UNSAT; A: SAT and preferred; C: SAT but incomplete

For a selected candidate a , the solver checks $\Phi_C \wedge B(a)$, where Φ_C is the compiled contract and $B(a)$ fixes its measurements and completion flag. SAT means compatible with this contract; UNSAT means incompatible. Independently implemented task rules compute admissibility and choose the admissible candidate with greatest progress. Agreement with that choice defines accuracy; selecting an inadmissible candidate defines a violation. Consequently, a SAT candidate can still be an accuracy error because it makes less progress or fails to complete. The two implementations share supplied task assumptions, so their agreement checks implementation correspondence rather than independent road safety.

The experimental section follows the same order: task-wise accuracy and violations, response to changed contracts, and specification/CNL quality. Detailed per-candidate records and unsupported-error cases remain in the supplement.

V. EXPERIMENTAL SETUP

A. EVALUATION OBJECTIVES AND SETUP

We distinguish the behavioral effects of adding explicit constraints to CoC from the checkability of the EBLC-to-CNL development path. P1 and P2 are both our constraint-enhancement approaches. P1–P2 is an internal comparison of development paths, not a contest against an external method.

The conditions share images, candidates, reference labels,

and splits. P1 and P2 receive constraints during training and evaluation; the P2 model receives CNL, not solver verdicts. Different contracts can assign different reference actions to an otherwise identical input, so P0 lacks information required to distinguish the pair. This evaluates supplied-constraint use rather than a causal training benefit under identical information.

We use Qwen3-VL-2B-Instruct with LoRA and seeds 42, 17, and 123. Within each task family, all arms use the same examples, three epochs, optimizer settings, and final-checkpoint policy. Temporal, static, and maneuver runs respectively use 960, 576, and 384 training rows and 180, 108, and 72 optimizer updates per fit. LoRA rank/alpha/dropout are 8/16/0.05; batch size is 2 with accumulation 8, AdamW learning rate 0.0002, and weight decay 0.01. Candidate scalars are supplied in text. The temporal family uses 40/10/10 distinct training/validation/test images; the static test reuses schematic images, and the maneuver family uses two repeated diagrams. Rows are not independent road scenes. Supplement S1 and S8 give split inventories and budgets.

The main metrics are **accuracy** against the highest-progress admissible candidate, **violation rate**, and **contract-pair accuracy**, which requires both members of a pair to be correct. An admissible but lower-progress choice is inaccurate without violating the contract. Tables show three-seed

TABLE 8. Comparison conditions and roles

Condition	Information added to CoC	Evaluation role
P0	No additional constraint	CoC-only baseline
P1	Existing natural-language behavioral constraint	Our preceding constraint-enhancement approach
P2	CNL generated after specifying and checking the same rule in EBLC	Formally supported constraint-development path

means; the main performance figure also shows individual seeds. Compliant completion is auxiliary and is not independent evidence when it equals one minus violation rate.

VI. RESULTS

A. PERFORMANCE ACROSS BEHAVIORAL TASKS

The six task rows correspond directly to Table 7: each prediction selects a candidate whose stored measurements are bound to that task's logical condition. Independent task rules determine admissibility and the reference choice; SMT checks the same bound candidate separately. Specification checks and prediction metrics therefore have different denominators.

P1 and P2 have higher accuracy and lower violation rates than P0 for all six task types. Temporal accuracy gains over P0 are 46.94 and 48.19 percentage points for P1 and P2, with violation reductions of 17.78 and 18.33 points. P1 and P2 tie in the three static types and stopping. Residual P2 lane-change errors select admissible but lower-progress candidates; compliant completion is 100%. Temporal P2 compliant completion is 99.17%.

P2 does not exceed P1 under every seed and condition. We report the effects of both paths relative to P0; formal noninferiority between P1 and P2 is not tested.

B. CONTRACT CHANGES AND CORRECT ASSOCIATION

Contract-pair accuracy is 0% for P0 in all three task families. In temporal, static, and maneuver families, respectively, P1 achieves 93.61%, 100%, and 100%, and P2 achieves 96.11%, 100%, and 97.92%. Constraint-conditioned models can change their selections when the rule changes while the visual input remains fixed. P0's low values must be interpreted with its missing contract information.

Shuffling training-time case-constraint associations while restoring correct constraints at evaluation gives accuracies of 50.56%, 50.69%, and 49.65%. Temporal shuffling uses natural-language guards, whereas new static and maneuver shuffling permutes P2 CNL within subtype; these are not pooled as one identical intervention. The results support the importance of correct case-constraint association beyond adding sentences alone.

In one recorded prediction with clearance at 11.0 s and a 1.5 s wait, P0 chooses entry at 11.5 s, while P1 and P2 choose the admissible boundary at 12.5 s. Another P2 example enters 0.1 s too early. These post-hoc explanatory cases are detailed in Supplement S5.

C. CHECKABILITY OF SPECIFICATIONS AND CNL

We inject errors involving negation, thresholds, release conditions, comparison direction, and units into the schema, evidence, Core, and CNL checking path. Supplement S3 reports the six supported types and their checking layers. This quality evaluation is distinct from model accuracy.

Detection across all 168 errors is 85.71%. These are multilayer checks, not the result of SAT solving alone. Errors shared by trusted evidence and specification can remain undetected; satisfiability and factual evidence validity are distinct.

All 100 temporal boundary queries match expected outcomes: 76 SAT, 24 UNSAT, and no UNKNOWN. Additional candidate checks and comparisons with model judgments appear in Supplement S8.

UNSAT here excludes a prohibited action rather than counting a defective specification. By contrast, increasing an injected wait by 100 ms makes an originally permitted query UNSAT, while removing the release restriction makes a prohibited query SAT. Unexpected outcomes require reviewing the source, threshold, and clause, then rechecking both sides of the boundary. An inconsistent base contract or UNKNOWN is not accepted as verification evidence. Supplement S3 retains the representative conflict set and response rules.

D. UNSEEN CONDITIONS AND INTERPRETATION

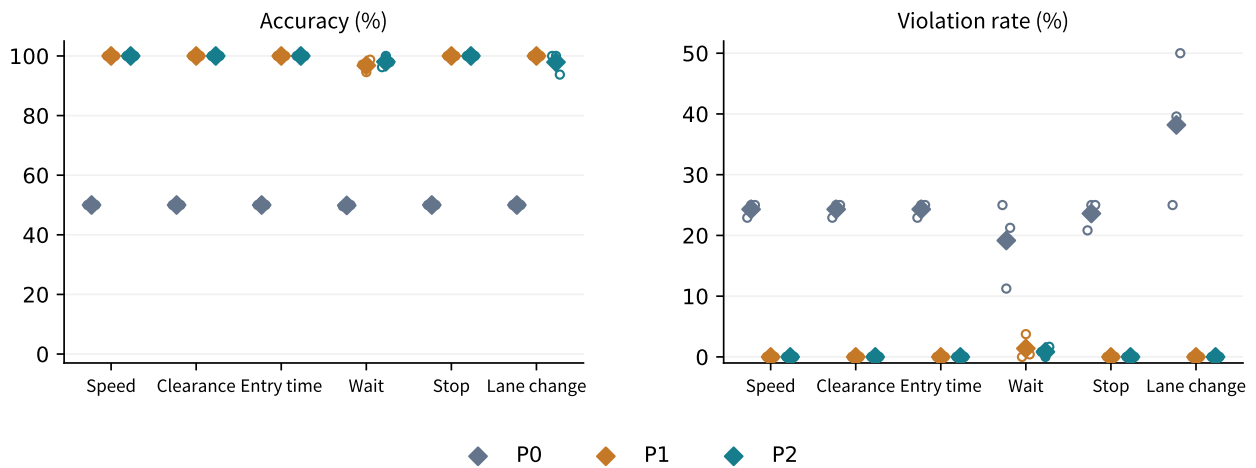
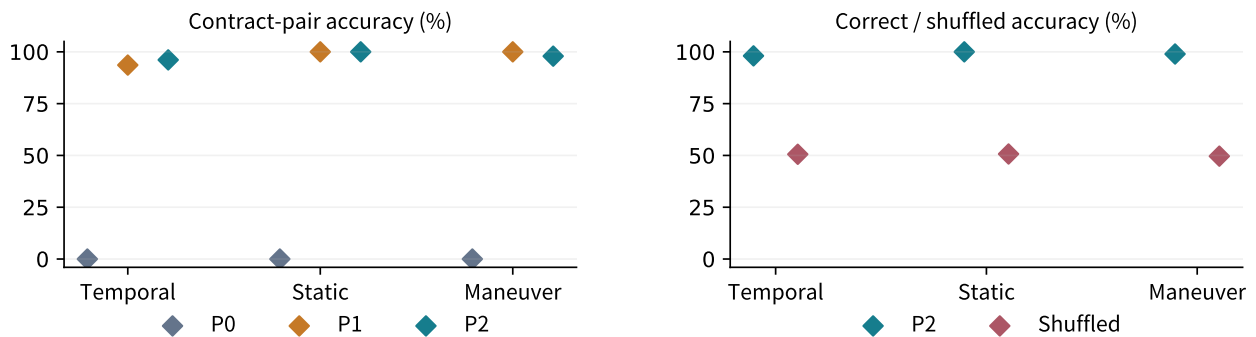
Both constraint-enhancement paths outperform P0 on unseen numeric/time conditions. Sensitivity to new wording differs by task: static paraphrasing gives P2 accuracy of 80.32% and violation rate of 7.87%. Paraphrasing, combined hard conditions, image interventions, and specification-derived auxiliary-cost learning are secondary diagnostics; favorable and unfavorable outcomes are retained in the supplement.

The results concern supplied synthetic constraints and finite candidates. Textual scalars, repeated diagrams, one model, and three seeds limit claims about real-road visual understanding, closed-loop safety, or Alpayo training. Constraint availability and the causal effect of verification are distinguished. Detailed variability, conditional intervals, and archived operational-criterion outcomes remain in the supplement.

Overall, multiple action tasks show benefits from constraint-enhanced CoC, while the EBLC-supported development path combines high action-selection performance with explicit checks of constraint errors and admissible/prohibited boundaries.

TABLE 9. Primary accuracy / violation rate, %, mean over three seeds

Task	P0	P1	P2
Maximum speed	50.00 / 24.31	100.00 / 0.00	100.00 / 0.00
Minimum clearance	50.00 / 24.31	100.00 / 0.00	100.00 / 0.00
Minimum entry delay	50.00 / 24.31	100.00 / 0.00	100.00 / 0.00
Entry after clearance and waiting	49.86 / 19.17	96.81 / 1.39	98.06 / 0.83
Stopping	50.00 / 23.61	100.00 / 0.00	100.00 / 0.00
Lane change	50.00 / 38.19	100.00 / 0.00	97.92 / 0.00

**FIGURE 6.** Accuracy and violation rate by task. Large points are means and small points are individual seeds. Zero observed violations refer to the evaluated samples.**FIGURE 7.** Left: contract-pair accuracy. Right: action accuracy of P2 with correct associations versus shuffled training associations. Points are three-seed means; static and maneuver values aggregate their subtypes.**TABLE 10.** Specification and CNL checks

Input category	Count	Outcome
Six supported error types	144	144 detected; 0 missed
Shared evidence–specification errors	24	0 detected; 24 missed
Normal contracts	24	24 accepted; 0 false alarms

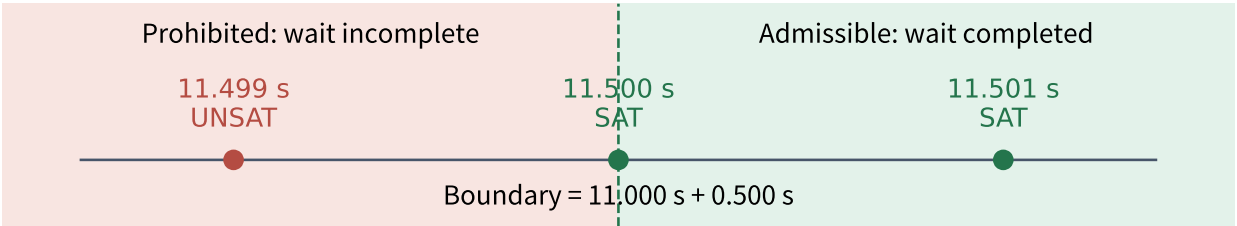


FIGURE 8. With clearance at 11.000 s and a 0.500 s wait, entry at 11.499 s is UNSAT; 11.500 s and 11.501 s are SAT. The 1 ms logical-time expansion does not imply vehicle-control precision.

TABLE 11. Unseen numeric/time conditions: accuracy / violation rate, %, three-seed means

Task family	P0	P1	P2
Temporal	49.86 / 18.47	96.67 / 2.22	96.67 / 1.53
Static	50.00 / 22.92	99.54 / 0.00	97.22 / 0.23
Maneuver	50.00 / 30.90	100.00 / 0.00	98.96 / 0.00

VII. DISCUSSION

A. THE ROLE OF EXPLICIT BEHAVIORAL CONSTRAINTS IN CoC

Whereas CoC connects a situation with reasons for an action, explicit behavioral constraints specify conditions that permit or restrict choices. The higher accuracy and lower violation rates of P1 and P2 relative to P0 across static, temporal, and maneuver tasks support this complementary role. Contract-pair evaluation is particularly informative: the same scene and candidates can require different choices under different

rules. Constraint enhancement therefore supplements the explanation in CoC with explicit conditions for action selection, rather than replacing that explanation.

The weak performance of shuffled associations indicates that adding constraint sentences alone is insufficient; their correspondence with each case matters. This supports the methodological motivation to connect targets, regions, and times to observations while distinguishing observations from applicable rules. However, P0 does not receive the variable contract, so the observed gains include the effect of supplying additional information. They should not be generalized to an overall deficiency in CoC reasoning or to greater learning efficiency under identical information.

B. FORMAL SUPPORT FOR CONSTRAINT DEVELOPMENT

This development path specifies which constraints to obtain, where their content comes from, and where it enters learning data. CoC contributes relevant actions and rationales; observations contribute targets, zones, and events; supplied rules contribute bounds and release requirements. These are linked to specification fields and sources before checked CNL is appended to the original CoC. Table 7 makes the task–field–formula–CNL–verdict chain concrete. The present contribution is this traceable, checkable development process, building on the preceding manuscript’s constraint-enhancement concept and evaluation principle.

P1 and P2 are not competing external methods but different development paths within our constraint-enhancement approach. P1 directly supplies existing natural-language constraints; P2 specifies and verifies the corresponding rules in EBLC before generating CNL. The central value of formalization is not necessarily higher model performance. It is the ability to distinguish targets, scope, activation, maintenance and release conditions, restricted actions, and assumptions in a checkable representation. Natural-language constraints can also be reviewed by people; the EBLC path additionally makes specification, translation, and generated-language correspondence explicit objects of supported checks.

Behavioral evaluation and error injection support different aspects of this value. The former shows that CNL produced through the formalization path can support high action-selection performance; the latter demonstrates checks for omissions, boundary errors, and semantic mismatches. Differences between P1 and P2 varied across tasks and expressions, so universal equivalence or noninferiority is not established. The supported claim is that systematic constraint development and checking can accompany the benefits of constraint enhancement, not that verification itself causes the behavioral improvement.

C. EVIDENCE, VERIFICATION, AND COMPLIANCE

Reliability in this process has three distinct layers. Evidence validity concerns whether the selected observations, rules, and assumptions are appropriate for the scene. Representation and translation validity concern whether the specification and CNL express the intended content. Behavioral compli-

ance concerns whether the model selects an action consistent with the supplied constraint. SMT verification checks logical properties under the specified model and assumptions, while supported correspondence checks examine specification-to-language agreement. Neither substitutes for establishing the factual validity of scene interpretation or rule evidence.

Undetected shared errors in evidence and specification expose a limitation of the first layer; residual model violations despite a correct specification expose a limitation of the third. These require different responses. Expected UNSAT when a prohibited action is conjoined with the contract supports its exclusion, whereas unexpected UNSAT for an admissible action motivates re-examining evidence, clauses, and translation. Specification verification and behavioral evaluation are thus complementary sources of evidence addressing different errors, rather than interchangeable assurances.

D. SCOPE AND FURTHER DEVELOPMENT

The behavioral evidence is limited to supplied synthetic rules, finite candidates, one VLM, and three optimization seeds. Candidate values are also provided in text, and some images are reused; performance on unseen values therefore does not establish visual generalization to new real-road scenes. P1 and P2 also receive constraints at evaluation, leaving internalization without supplied constraints unexamined. Sensitivity to paraphrasing describes the applicability of the model and its input interface; this study does not isolate underlying language capability as its sole cause. Error detection likewise remains bounded by supported syntax and injected error types.

Further work can evaluate how constraint applicability is established in real scenes and how generated CNL is used across models and environments. A promising division of work concentrates human review on observations, rules, and assumptions, while assigning structuring, specification checks, and language generation to machine processing. GuardSynth Studio can provide an implementation setting for this workflow, but extraction accuracy and reductions in authoring time require separate evaluation. The present study establishes a more limited foundation: evidence for constraint-enhanced CoC action selection and a systematic path from structured, checked constraints to natural-language learning inputs.

VIII. CONCLUSION

GuardSynth provides a systematic development path for behavioral constraints that augment Chain of Causation. It identifies constraint content and its sources, binds it to supported EBLC fields and semantics, checks a restricted logical representation, and generates CNL for insertion after the original CoC. The task-field-formula-CNL-verdict mapping connects this method to executable examples and evaluation.

Across six synthetic candidate-selection tasks, our existing natural-language constraint path and the EBLC-derived CNL path achieve higher primary accuracy and lower violation rates than CoC alone. Separate error and boundary analy-

ses demonstrate supported checks while exposing shared-source errors and residual model violations. The contribution combines the behavioral utility of explicit constraints with a traceable formal development interface; broader extraction reliability and real-road generalization remain future work.

ACKNOWLEDGMENT

OpenAI Codex assisted drafting, translation, editing, and formatting of Sections I–VIII. The authors remain responsible for checking claims and references. Author, affiliation, funding, and prior-publication metadata require final confirmation.[15]

REFERENCES

- [1] NVIDIA, “Alpamayo-R1: Bridging Reasoning and Action Prediction for Generalizable Autonomous Driving in the Long Tail,” arXiv:2511.00088, 2025, revised 2026. <https://arxiv.org/abs/2511.00088>
- [2] C. Sima et al., “DriveLM: Driving with Graph Visual Question Answering,” ECCV, 2024. <https://arxiv.org/abs/2312.14150>
- [3] X. Tian et al., “DriveVLM: The Convergence of Autonomous Driving and Large Vision-Language Models,” 2024. <https://arxiv.org/abs/2402.12289>
- [4] Research team’s preceding manuscript, Guard-Aware Trajectory Candidate Selection for Vision-Language Autonomous Driving: Safety-Constrained Chain-of-Causation, supplied revision v01. Publication details pending author confirmation.
- [5] Y. Li et al., “Drive-R1: Bridging Reasoning and Planning in VLMs for Autonomous Driving with Reinforcement Learning,” AAAI, vol. 40, no. 8, pp. 6708–6716, 2026. <https://ojs.aaai.org/index.php/AAAI/article/view/37602>
- [6] K. Esterle, L. Gressenbuch, and A. Knoll, “Formalizing Traffic Rules for Machine Interpretability,” IEEE CAVS, 2020. <https://arxiv.org/abs/2007.00330>
- [7] T. Cai et al., “Driving with Regulation: Trustworthy and Interpretable Decision-Making for Autonomous Driving with Retrieval-Augmented Reasoning,” AAAI, vol. 40, no. 45, pp. 38287–38295, 2026. <https://ojs.aaai.org/index.php/AAAI/article/view/41168>
- [8] Y. Chen, R. Gandhi, Y. Zhang, and C. Fan, “NL2TL: Transforming Natural Languages to Temporal Logics using Large Language Models,” EMNLP, pp. 15880–15903, 2023. <https://aclanthology.org/2023.emnlp-main.985/>
- [9] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, and C. Trippel, “nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models,” CAV, 2023. <https://arxiv.org/abs/2303.04864>
- [10] N. E. Fuchs, K. Kaljurand, and G. Schneider, “Attempto Controlled English Meets the Challenges of Knowledge Representation, Reasoning, Interoperability and User Interfaces,” 2006. <https://attempto.ifi.uzh.ch/site/pubs/papers/FLAIRS0601FuchsN.pdf>
- [11] C. Barrett and C. Tinelli, “Satisfiability Modulo Theories,” Handbook of Model Checking, pp. 305–343, 2018. <https://theory.stanford.edu/~barrett/pubs/BT18.pdf>
- [12] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” TACAS, pp. 337–340, 2008. https://doi.org/10.1007/978-3-540-78800-3_24
- [13] T. Kuhn, “A Survey and Classification of Controlled Natural Languages,” Computational Linguistics, vol. 40, no. 1, pp. 121–170, 2014. <https://aclanthology.org/J14-1005/>
- [14] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [15] OpenAI, Codex. Software used for drafting, translation, editing, and formatting assistance. <https://openai.com/codex/>

...